Hospital Management System

Executive Architecture & Software Documentation Manual

1. Executive Summary

The Hospital Management System is a comprehensive platform for managing healthcare operations.

2. Project Vision

The Hospital Management System is a comprehensive platform for managing healthcare operations. Designed as a production-ready software solution, the system emphasizes enterprise scalability, high availability, and robust security serving Admin, Doctor, Nurse, Patient across client applications.

3. Functional Requirements

- Patient Registration
- Doctor Scheduling
- Lab Results
- Pharmacy Billing

4. Non-Functional Requirements

- HIPAA Compliance
- 99.9% Uptime
- Sub-second API response

5. User Roles

Configured Roles: Admin, Doctor, Nurse, Patient

6. Use Cases

UC-01: EHR Management

Actor: Admin

Preconditions: Admin is authenticated with valid authorization token

Main Flow:

1. Admin accesses the application interface
2. System validates input parameters and user permissions
3. Client dispatches request payload to method='GET' path='/api/v1/patients' description='List patients'
4. Backend executes business logic transaction and persists updates to database
5. System returns status confirmation and renders updated view for Admin

Postconditions: Transaction committed to database and state synchronized

UC-02: Appointment Booking

Actor: Doctor

Preconditions: Doctor is authenticated with valid authorization token

Main Flow:

1. Doctor accesses the application interface
2. System validates input parameters and user permissions
3. Client dispatches request payload to method='POST' path='/api/v1/appointments' description='Create appointment'

4. Backend executes business logic transaction and persists updates to database
5. System returns status confirmation and renders updated view for Doctor

Postconditions: Transaction committed to database and state synchronized

UC-03: Billing & Invoicing

Actor: Nurse

Preconditions: Nurse is authenticated with valid authorization token

Main Flow:

1. Nurse accesses the application interface
2. System validates input parameters and user permissions
3. Client dispatches request payload to method='GET' path='/api/v1/patients' description='List patients'
4. Backend executes business logic transaction and persists updates to database
5. System returns status confirmation and renders updated view for Nurse

Postconditions: Transaction committed to database and state synchronized

7. Recommended Tech Stack

Technologies: React, FastAPI, PostgreSQL, Docker

8. Database Design

```
[ Database ER Diagram ]
erDiagram
  PATIENT ||--o{ APPOINTMENT : has
  DOCTOR ||--o{ APPOINTMENT : conducts
```

Table: name='patients' purpose='Stores patient profiles' columns=['id: int', 'name: str', 'dob: date'] ()

Columns:

Table: name='appointments' purpose='Doctor bookings' columns=['id: int', 'patient_id: int', 'doctor_id: int', 'scheduled_at: datetime'] ()

Columns:

9. API Documentation

```
[ API Sequence Diagram ]
sequenceDiagram
  actor Patient
  participant API as Backend API
  participant DB as Database
  Patient->>API: POST /api/v1/appointments
  API->>DB: INSERT appointment
  DB-->>API: OK
  API-->>Patient: 201 Created
```

- [GET] method='GET' path='/api/v1/patients' description='List patients':
- [GET] method='POST' path='/api/v1/appointments' description='Create appointment':

10. Folder Structure

```
backend/
  app/
    api/
    models/
frontend/
  src/
```

11. System Architecture

The system architecture utilizes a multi-tier decoupled topology built on React, FastAPI, PostgreSQL, Docker. The presentation layer communicates with the backend via 2 REST API endpoints, persisting state across 2 relational database tables. Security recommendations include JWT Auth, TLS 1.3, RBAC.

```
[ System Architecture Diagram ]
flowchart TD
  Client["Web / Mobile Client"] --> API["FastAPI Backend"]
  API --> DB[("PostgreSQL Database")]
```

12. Deployment Strategy

```
[ Deployment Diagram ]
flowchart LR
  Users --> CDN[Cloudflare CDN]
  CDN --> LB[Load Balancer]
  LB --> App1[App Instance 1]
  LB --> App2[App Instance 2]
  App1 --> RDS[(PostgreSQL RDS)]
```

- Docker Containers
- Kubernetes
- AWS RDS

13. Development Timeline

```
[ Application Workflow Diagram ]
flowchart TD
  Start([Start]) --> Login[User Login]
  Login --> Dashboard[View Dashboard]
  Dashboard --> Action[Book Appointment]
```

- Phase 1: MVP
- Phase 2: Billing & Pharmacy

14. Future Enhancements

- [High Impact] Security Hardening: JWT Auth: JWT Auth
- [High Impact] Security Hardening: TLS 1.3: TLS 1.3
- [High Impact] Security Hardening: RBAC: RBAC
- [High Impact] Microservices Decoupling & Global Edge Caching: Decouple high-traffic API modules into microservices with global CDN edge caching.
- [Medium Impact] Real-Time Telemetry & Distributed Tracing: Integrate OpenTelemetry collectors and automated operational monitoring dashboards.